

Hackathon Jury Q&A Prep Script:

Mechatronician Hub

This script contains **50 realistic, high-pressure questions** a hackathon jury might ask about the **Mechatronician Hub**, categorized by focus area. Each question is paired with a strategic, high-impact answer designed to demonstrate technical depth, commercial viability, and user empathy.

Table of Contents

1. **Problem Statement & Value Proposition (Q1–Q10)**
 2. **Product Experience & Features (Q11–Q20)**
 3. **Technical Architecture & Database (Q21–Q30)**
 4. **Trust, Security & Moderation (Q31–Q40)**
 5. **Scale, Viability & Future Vision (Q41–Q50)**
-

1. Problem Statement & Value Proposition

Q1: What is the core problem you are trying to solve?

- **Answer:** We are solving the "**friend-circle limitation**" in colleges. Most students form project or hackathon teams exclusively within their immediate group of friends, leading to skill-monotonous teams (e.g., all frontend developers, or all mechanical engineers). Mechatronian Hub acts as a specialized, cross-disciplinary matchmaking engine that connects students based on complementary skills, interests, and project needs rather than social circles.

Q2: Why can't students just use Discord or WhatsApp groups to find teammates?

- **Answer:** Social channels like WhatsApp and Discord are chaotic, unorganized, and rely on real-time active scrolling. A post looking for a teammate gets buried in minutes. Mechatronian Hub is a structured directory: listings are persistent, searchable by specific tech stack, categorized by roles needed, and linked directly to verified profiles. We offer **structured matchmaking**, not a chaotic chat feed.

Q3: What is the exact target demographic?

- **Answer:** Our primary demographic is college students—specifically engineering, design, and management students—who are actively working on capstone projects, lab assignments, research papers, and national/global hackathons.

Q4: How did you validate that this is a real problem?

- **Answer:** We validated this through user research and our own experiences. In average colleges, up to 75% of high-potential students struggle to find skilled partners outside their branch (e.g., a CS student finding a Mechanical student for a robotics project). By speaking to peers and coordinators, we verified that team formation is the single biggest bottleneck preventing students from entering hackathons.

Q5: What makes Mechatronian Hub different from LinkedIn?

- **Answer:** LinkedIn is built for professional networking and career search on a global scale. It is too broad and noisy for immediate, localized peer collaboration. Mechatronian Hub is localized to the student ecosystem, highly action-oriented (e.g., immediate "recruitment posts" for an upcoming hackathon next Friday), and integrates specific academic branch/semester tags that LinkedIn lacks.

Q6: Can you explain the "Mechatronian" branding?

- **Answer:** "Mechatronian" represents the convergence of multi-disciplinary engineering—combining Mechanical, Electronics, and Computer Science. It symbolizes the spirit of modern, cross-functional collaboration where software developers work side-by-side with hardware builders and product designers.

Q7: If a student is shy or introverted, how does your app help them?

- **Answer:** Shy students often struggle to approach peers in person or self-promote in massive group chats. Our platform lowers the activation barrier by offering a structured "Apply to Team" button where they can express interest privately and share their portfolio with one click, taking away the social anxiety of pitching themselves verbally.

Q8: How does this benefit the college administration?

- **Answer:** It acts as a centralization tool for the college. Faculty coordinators can view ongoing projects, broadcast hackathons directly to the student dashboard, monitor collaboration trends, and identify high-performing teams for national representations.

Q9: Does this platform replace face-to-face networking, or supplement it?

- **Answer:** It supplements it. Mechatronian Hub is the discovery layer. Once students identify and verify each other's skills and interests on the hub, they meet in campus labs, hackathons, and coffee shops to build. We bridge the initial discovery gap.

Q10: How do you define a "successful match" on your system?

- **Answer:** A match is defined as a accepted Team Request where a listing owner accepts a applicant's request, shifting their status from "Pending" to "Joined" in the team workspace.

2. Product Experience & Features

Q11: Walk me through the user flow for a student wanting to join a hackathon team.

- **Answer:**

1. The student logs in and navigates to the **Teams** tab.

2. They filter listings by skills wanted or target hackathon.

3. They click on a listing to view details, roles open, and the team size.

4. They click "Request to Join" and optionally add a short pitch.

5. The Team Leader receives a real-time notification, views the applicant's verified profile, and clicks "Accept".

6. Both are placed in a shared team chat workspace to start building.

Q12: How do you handle team recruitment listings that are no longer active?

- **Answer:** Listings have an expiration date tied to the hackathon or project deadline. Once reached, they are archived. Additionally, creators can mark their listing as "Filled", which visually disables applicants while preserving the team structure for portfolio tracking.

Q13: What happens when someone clicks on the WhatsApp icon on a user's profile?

- **Answer:** It leverages a direct API redirection (<https://wa.me/number>) with a pre-filled, professional greeting: *"Hi, I saw your profile on Mechatronian Hub and would love to collaborate on..."*. This provides smooth, low-friction off-platform communication.

Q14: How does the "Live Activity Feed" work, and what is its purpose?

- **Answer:** The Activity Feed provides micro-interactions that make the platform feel alive. It broadcasts anonymous, real-time events like *"John just posted a new AI project"* or *"A team was formed for Smart India Hackathon"*. This psychological trigger encourages continuous user engagement.

Q15: Why did you decide to implement a custom PWA (Progressive Web App) instead of a native mobile app?

- **Answer:** Mobile friction is high; students hate downloading 50MB native apps from the app store for internal college use. As a PWA, Mechatronian Hub is fast, operates on a lightweight footprint, can be installed directly from Safari or Chrome as a desktop/home screen icon, and supports offline access—saving time and storage.

Q16: How do you manage user notifications?

- **Answer:** We implement dual-layered notifications:

1. **In-App Notifications:** Real-time toast alerts driven by Supabase realtime subscriptions.

2. **Push Notifications:** PWA-integrated notifications via OneSignal, ensuring users receive team invitations and chats even when the app is closed.

Q17: Can you explain the role of the "Coordinator" on the dashboard?

- **Answer:** Coordinators (often faculty or club leaders) have elevated permissions. They can create official "Events" (like an upcoming college-wide ideathon), view analytics of project categories, and assist in manual team balancing if some students are left without partners.

Q18: What filters are available for search, and how do they work?

- **Answer:** Students can filter teams by **Skills required** (React, IoT, CAD, etc.), **Project Type** (Academic, Hackathon, Passion Project), and **Branch/Specialization**. The search is instant, updating the UI dynamically without page reloads.

Q19: How do team chats operate? Are they persistent?

- **Answer:** Yes, team chats are persistent. When a team is formed, a dedicated chat channel is generated in the database. Messages are stored securely and delivered in real-time to active team members using secure web sockets.

Q20: Why did you choose a dark-themed glassmorphic design system?

- **Answer:** Hackathon builders and developers overwhelmingly prefer dark mode as it reduces eye strain during long-night coding sessions. The glassmorphism and subtle gradients provide a premium, modern aesthetic that stands out and feels high-quality immediately.

3. Technical Architecture & Database

Q21: What is your technology stack, and why did you choose it?

- **Answer:** Our stack is built for rapid development, scale, and responsiveness:
- **Frontend:** React 19 + Vite (for high-speed bundle compile, performance, and modern SPA capabilities).
- **Styling:** Vanilla CSS3 custom variables (highly responsive glassmorphic design system with no heavy UI framework overhead).
- **Backend-as-a-Service:** Supabase (PostgreSQL database, Row-Level Security, built-in GoTrue authentication, and native WebSockets for instant chat).
- **Hosting:** Vercel (Edge-network global caching and instant CI/CD deployment pipeline).

Q22: Why did you choose Supabase over a standard Node/Express backend with MongoDB?

- **Answer:** For a hackathon timeline, speed-to-market is key. Supabase gave us production-grade security, instant APIs generated directly from our PostgreSQL database schema, and native Realtime WebSockets out of the box. PostgreSQL's relational nature is also vastly superior to MongoDB for matching profiles, listings, and requests through structured foreign-key relationships.

Q23: How is your database structured?

- **Answer:** We designed a highly normalized relational schema:
- **profiles:** Stores student data, branch, bio, and verification states. Linked 1:1 with Supabase Auth.
- **team_listings:** Stores project descriptions, skills needed, and owner IDs.
- **team_requests:** Manages application states (Pending, Accepted, Rejected) linking profiles to listings.
- **chats:** Handles message persistence for accepted team channels.

Q24: How does your application handle real-time state synchronization?

- **Answer:** We leverage Supabase's PostgreSQL Replication capabilities. The frontend listens to changes on target tables (like **chats** or **team_requests**) via standard WebSocket connections. When a database row is inserted, the backend pushes the delta payload to the client, triggering an immediate, seamless UI update.

Q25: How do you optimize page load speeds in your SPA?

- **Answer:**

1. We implement React **code-splitting** using `React.lazy` and `Suspense` so dashboards and admin panels only load when navigated to.
2. Assets are compressed and cached globally on Vercel's Edge CDN.
3. The client utilizes a single, lightweight SVG icon sprite sheet (`icons.svg`) to avoid multiple HTTP requests for visual elements.

Q26: What was the recent bug with your Vercel routing configuration, and how did you resolve it?

- **Answer:** Since our app is an SPA, all virtual routes must rewrite to `index.html`. Initially, a broad rewrite `/(.*)` was capturing critical static files (`sitemap.xml`, `robots.txt`, GSC HTML verification file) and rendering them as HTML instead of their native formats. We resolved this by implementing an advanced negative lookahead regex rewrite in `vercel.json` to explicitly exclude static assets and search engine files, ensuring clean, direct filesystem access.

Q27: How does your database prevent orphaned records when a user deletes their account?

- **Answer:** We set up strict **Foreign Key Constraints** in PostgreSQL with **ON DELETE CASCADE** rules. For example, if a profile is deleted, all their pending **team_requests** and direct **chats** are automatically and cleanly purged by the database engine to maintain integrity.

Q28: How do you handle file uploads, like profile avatars or project banners?

- **Answer:** We use **Supabase Storage buckets**. Uploaded files are run through a client-side image compression step, saved to a secure public folder in Supabase, and the resulting unique CDN URL is saved in the user's **profiles** database row.

Q29: Is there any state management library used, or how do you propagate state?

- **Answer:** We avoid unnecessary boilerplate like Redux. Instead, we use **React's built-in State and Effect hooks**, driving global auth session states from the root **App** component down to child views, resulting in an exceptionally fast, highly predictable data flow.

Q30: How does your application deal with offline access?

- **Answer:** Our Service Worker (`sw.js`) caches static shell assets (HTML, CSS, JS, logos) on the first visit. If a student loses internet access on campus, the app still launches immediately, rendering an elegant offline message and displaying cached team structures from the browser's local state.

4. Trust, Security & Moderation

Q31: How do you prevent malicious actors or students from other colleges from spamming your system?

- **Answer:** Our registration system has built-in verification gates. We restrict access strictly to students with a valid academic email domain. Additionally, we've integrated strict **Row-Level Security (RLS)** policies in PostgreSQL, meaning users can only write data if they possess an authenticated JWT session token from our system.

Q32: Can you explain your LinkedIn Verification feature?

- **Answer:** To ensure that skills and portfolios listed by students are genuine and professional, we integrated LinkedIn OAuth. When a student verifies their profile, our system cross-checks their identity and displays a secure green "LinkedIn Verified" checkmark on their profile page. This builds a trustworthy marketplace where team leaders know they are recruiting real, capable talent.

Q33: How do you handle cyberbullying, harassment, or spam in team chats?

- **Answer:** We have implemented a two-part moderation framework:
 1. **Block Feature:** Users can block abusive peers, which instantly hides their chat messages and prevents them from applying to their team listings.
 2. **Report System:** A one-click flag system that logs complaints into an `admin_reports` table, prompting a notification on the Admin Dashboard for immediate moderator intervention.

Q34: What is Supabase Row-Level Security (RLS), and how does it protect user data?

- **Answer:** RLS ensures that a user cannot manipulate or read another user's database records. For example, we wrote an RLS policy for the `chats` table that reads: `"Allow read access only if the authenticated user's ID matches one of the members of that team channel"`. This security is enforced directly by PostgreSQL, preventing database exploits even if the client-side code is compromised.

Q35: How do you protect a user's phone number or WhatsApp details?

- **Answer:** Contact details are private by default. A student's WhatsApp contact button is only revealed *after* a Team Request has been officially accepted by the team owner, ensuring no one can scrape student phone numbers from the directory.

Q36: What permissions does the Administrator have, and how is it secure?

- **Answer:** Admins have a dedicated dashboard that bypasses standard limits, powered by database triggers that assign a `role = 'admin'` metadata field to select accounts. Admins can suspend accounts, delete listings that violate terms of service, and resolve flagged chats.

Q37: How do you prevent SQL Injection attacks?

- **Answer:** By using Supabase's JavaScript Client API, all database queries are converted into parameterized HTTP requests processed by **PostgREST**. This completely eliminates the risk of raw SQL query manipulation and injection.

Q38: How do you manage password hashing and authentication security?

- **Answer:** User authentication is handled entirely by Supabase Auth (GoTrue), which uses industry-standard bcrypt hashing for passwords and issues encrypted, short-lived JWT (JSON Web Tokens) to manage sessions securely.

Q39: Can an administrator view private team chats without being a member?

- **Answer:** No. To respect user privacy, only official "reported" messages copied to the moderation log can be viewed by admins. Raw team chats are locked behind RLS policies restricted purely to active team members.

Q40: What happens if a blocked user tries to apply to a team?

- **Answer:** Our backend checks if a block relationship exists between the applicant and the listing owner. If true, the system throws a silent validation error, preventing the application request from ever reaching the owner's dashboard.

5. Scale, Viability & Future Vision

Q41: How do you plan to scale this platform to multiple colleges?

- **Answer:** Our database is architected with multi-tenant capabilities. We can introduce a `college_id` column across tables, allowing us to serve multiple campus campuses under a single deployment. This isolates campus directories while letting students collaborate across different universities for national hackathons.

Q42: What is your monetization strategy?

- **Answer:** While the core directory is free for students:

1. **Premium Recruitment:** Companies looking for specialized tech talent can pay to post official internship and job challenges directly to the student dashboard.

2. **Sponsorships:** Brands and hackathon organizers can buy "featured" spots on our events calendar.

Q43: How will you handle the platform when thousands of users are active simultaneously?

- **Answer:** Our frontend is entirely static, hosted on Vercel's global CDN which handles spikes effortlessly. Our database (Supabase/PostgreSQL) can scale up dynamically, and we can implement connection pooling (via PgBouncer) and database indexing on frequently searched columns like **skills** and **roles** to keep response times under 50ms.

Q44: Can you integrate AI or machine learning into this in the future?

- **Answer:** Absolutely. The future roadmap includes a **Smart Matchmaking Engine**. By analyzing past team successes, student grades, and skills, an AI recommendation system can suggest the top 3 ideal teammates for your project description using PostgreSQL pgvector similarity searches.

Q45: How will you keep users returning to the app after their hackathon is over?

- **Answer:** We transition their profile from "Hackathon Builder" to an "Alumni/Portfolio Showcase". Their successful projects remain visible on their profiles as verified work histories. This turns the platform into a living campus portfolio network, encouraging seniors to mentor juniors and developers to recruit new team members.

Q46: How do you encourage cross-branch collaboration?

- **Answer:** Our onboarding process explicitly highlights complementary skills. When a CS student creates a "Robotics" project, our onboarding flow guides them to seek Mechanical students, highlighting their skills as highly compatible and showing them verified profiles from other branches.

Q47: What is your plan for user acquisition in the first week?

- **Answer:** We will partner with college clubs (like the Google Developer Student Clubs or Robotics Club). By making it the official registration portal for the upcoming college hackathon, we can secure 100% of target participants in week one.

Q48: How do you prevent "zombie accounts" (users who register but never use the app)?

- **Answer:** We use automated email triggers. If a student is inactive for 2 weeks, our system sends a curated email digest highlighting active projects and team requests looking for their specific skill set, bringing them back into the collaboration funnel.

Q49: If you could add one major feature next, what would it be?

- **Answer:** An integrated "**Idea Playground**" where students can share raw, unformed ideas to gather community feedback via upvotes and comments before committing to a full team listing.

Q50: Why should the jury invest in or choose your project?

- **Answer:** Because Mechatronian Hub is a fully functional, highly secure, production-grade application that solves a genuine, painful bottleneck in the student innovation cycle. We don't just present a mockup; we have built a secure, real-time collaboration ecosystem that is ready to deploy on any campus tomorrow!

